

Support Vector Machines: Margin Maximization, KKT, and the Hinge

Yuval Lavie

August 10, 2026

Abstract

The SVM is the batch form of the margin idea: among all separating hyperplanes, pick the one farthest from the data; when separation is imperfect, trade margin width against violations. We derive the hard-margin program from the geometry, apply the KKT conditions of the optimization note to obtain the signature structural fact — the solution is carried by the *support vectors* alone — introduce the soft margin and its equivalent hinge-loss form, and optimize the latter with Pegasos (subgradient SGD). `svm.py` verifies the optimum, the support-vector property (by deletion experiments), and compares the boundary with logistic regression's.

1 Hard margin: the geometry

Labels $y_i \in \{-1, +1\}$, predictor $\text{sign}(w^\top x + b)$. The distance of x_i from the hyperplane $\{w^\top x + b = 0\}$ is $y_i(w^\top x_i + b)/\|w\|$ (positive iff correctly classified). Maximizing the smallest such distance is, after normalizing the scale freedom $(w, b) \rightarrow (cw, cb)$ by *fixing the closest point's functional margin to 1*, the program

$$\min_{w, b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i (w^\top x_i + b) \geq 1, \quad i = 1, \dots, n, \quad (1)$$

whose geometric margin is $1/\|w\|$ — minimizing the norm *is* maximizing the margin. A convex quadratic program with linear constraints: exactly the setting of the KKT theorem.

2 KKT: the support vectors

Proposition 1. *At the optimum of (1) with multipliers $\alpha_i \geq 0$,*

$$w^* = \sum_{i=1}^n \alpha_i y_i x_i, \quad \alpha_i > 0 \implies y_i (w^{*\top} x_i + b^*) = 1. \quad (2)$$

Proof. The Lagrangian is $\frac{1}{2} \|w\|^2 - \sum_i \alpha_i (y_i (w^\top x_i + b) - 1)$. Stationarity in w gives $w = \sum_i \alpha_i y_i x_i$; stationarity in b gives $\sum_i \alpha_i y_i = 0$; and complementary slackness $\alpha_i (y_i (w^\top x_i + b) - 1) = 0$ forces $\alpha_i = 0$ for every point with margin strictly above 1. $\square$

Points beyond the margin are *invisible* to the solution: only the *support vectors* — those on (or, later, inside) the margin — have $\alpha_i > 0$ and enter w^* . The script tests this operationally in the soft-margin form: removing every non-support point (keeping the original $1/n$ weighting, so the loss–regularizer balance is untouched — their hinge terms are identically zero near w^* , so the objective is locally unchanged) leaves the refitted optimum within subgradient noise of w^* , while removing the support vectors moves it by an order of magnitude more. Sparsity in *examples* is the SVM’s signature, exactly as complementary slackness promised in the optimization note.

3 Soft margin and the hinge form

Real data overlaps; (1) is then infeasible. Introduce slacks $\xi_i \geq 0$ measuring each violation and price them:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \quad (3)$$

At the optimum $\xi_i = \max(0, 1 - y_i(w^\top x_i + b))$ — the *hinge loss* — so (3) is equivalent to the unconstrained regularized risk

$$\min_{w,b} \frac{\lambda}{2}\|w\|^2 + \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(w^\top x_i + b)), \quad \lambda = \frac{1}{Cn}, \quad (4)$$

margin geometry re-expressed as *loss + regularizer* — the form the losses note studies, and the bridge from the constrained world of (1) to ordinary (sub)gradient training. KKT survives the generalization: support vectors are now the points with $y_i f(x_i) \leq 1$ (on or inside the margin, including violators), still typically a small fraction of the data.

4 Optimization: subgradients and Pegasos

The hinge is convex but has a kink at margin 1, so gradients give way to *subgradients*: any slope between the two one-sided slopes is valid at the kink, and subgradient descent retains convex convergence guarantees (at kink-free points the subgradient is the gradient; $-y_i x_i$ works on the violating side, 0 on the satisfied side). Pegasos is SGD on (4) with the schedule $\eta_t = 1/(\lambda t)$:

$$w_{t+1} = (1 - \eta_t \lambda) w_t + \eta_t y_{i_t} x_{i_t} \mathbf{1}[y_{i_t} w_t^\top x_{i_t} < 1], \quad (5)$$

a shrink step (the regularizer, pulling toward larger margin) plus a perceptron-style correction on margin violators only. Note the family resemblance across the last three notes: perceptron corrects mistakes by a fixed step; PA corrects margin violations by an optimized step; Pegasos corrects margin violations by a decaying step while shrinking — three tempos of the same drum. $\tilde{O}(1/(\lambda \varepsilon))$ iterations suffice for ε -accuracy, independent of n : the script’s Pegasos run matches a long full-batch subgradient reference to the third decimal, and random perturbation finds no improvement.

Remark 1 (SVM vs logistic regression). *On the script’s overlapping Gaussians the two boundaries nearly agree, but the philosophies differ: logistic regression models $\Pr(Y=1|x)$ and uses every point (weighted by $p(1-p)$); the SVM makes no probabilistic claim, outputs no calibrated probability, and listens only to the margin’s neighborhood. Hinge vs log-loss is the precise version of this contrast — see the losses note.*

Remark 2 (Kernels, in one paragraph). *By (2), both training (in the dual) and prediction ($w^{*\top}x = \sum_i \alpha_i y_i x_i^\top x$) touch the data only through inner products $x_i^\top x_j$. Replacing them by a kernel $k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$ runs the same linear machinery in an implicit feature space — nonlinear boundaries with no explicit feature map. That door, opened by the dual’s structure, is a note for another day.*